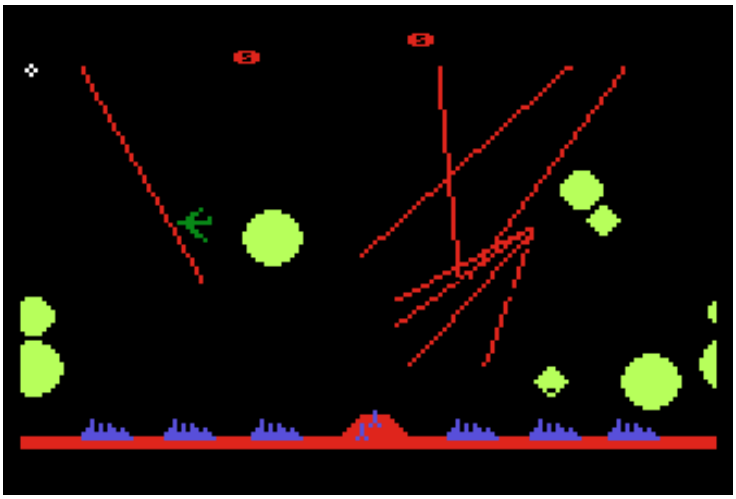
HW 03:

Extended Mode 3 Game



Purpose: To build a more complex game in Mode 3 to further your understanding of: structs, arrays, and object pooling.

Instructions

In this homework, you will be making a more complex game in Mode 3. This **must be something different than what you implemented for HW02 or any of the labs.** Do not recreate any examples shown for previous assignments and demos. Also, **you must use the provided scaffold as the basis for your homework.**

The design of this game must be more complex than *catching/dodging falling boxes* or basic *Missile Command*. We are leaving this one somewhat open-ended so that you can push yourself creatively and see what fun game you can come up with! You should adhere to the technical requirements and game mechanics listed below, but you can decide on the theme and any unspecified mechanics.



Requirements

Minimum Game Features

Your *game* must have the following:

- At least **two structs.**
- At least **one array.**
- At least **one use of DMA**.
    - Complete the DMANow function in the scaffold and replace the drawRectangle and fillScreen functions so they use DMA.
- Meaningful **object pooling.**
- A **state machine** including at least the following states:
    - Start,
    - Pause,
    - Game,
    - Win and/or Lose.
        - It is ok if your game is a survival-based game, and therefore only has a Lose state!
- You must be able to **navigate between the states** in some way (e.g. pressing the button START while in the Start state takes you to the Game state, meeting the win condition while in the Game state takes you to the Win state, etc.).
- At least **two types of moving objects**.
    - These *must use two different structs*. They must also *look* **and** *behave* differently (e.g. green player controlled by buttons and red enemy chasing player).
- **Collision that matters** (i.e. *something* must happen whenever two different objects touch each other—think bullet+enemy, enemy+player, tomato+face, etc.).
- At least **four buttons** used for input.
- At least **two different sound effects using analog sound.**
    - These sound effects should occur when something happens (e.g. collision with enemies, shooting bullets, losing a life, etc.).
    - These may be student-created or chosen from the premade library.
- A **README.md** file.
    - An instruction manual (of sorts) that tells a player how to play your game, including things such as:
        - What each button does (controls),
        - How to navigate your state machine,
        - How to win and/or lose your game,
        - Anything that is buggy or not completed.
    - If you have little to no experience with Markdown syntax, here is a cheat



sheet and a more thorough explanation.
    - You must **use at least one form of Markdown formatting correctly** (header, bold, italics, etc.) and this use must **make some sort of logical sense**.
- A **minimal amount of flicker.**
- **Polish and enjoyable gameplay**
    - Create a visually appealing, engaging experience that feels good or intuitive to play.

Extra Credit Considerations

Above and Beyond Gameplay Mechanic

For extra credit consideration, create an additional gameplay mechanic that goes above and beyond the main functionality of your game. For example, a game like *Missile Command* could have an additional mechanic that allows the player to shoot multiple bombs at once. In other words, the game must feel complete even without this mechanic; including this new element should only add to the gameplay experience.

You must implement this additional mechanic **and** describe why you think it goes above and beyond the standard gameplay in your README.md to receive **up to 10 pts of extra credit** to your assignment grade.

Code/files

Your *code* must have the following:

- At least **six .c files.**
    - The scaffold contains main.c, gba.c, font.c, analogSound.c, and print.c. This means you have to create at least one more source file for this assignment.
- At least **five .h files.**
    - The scaffold contains gba.h, font.h, analogSound.h, and print.h. This means you have to create at least one more header file for this assignment.
- Good organization (see tips below).
- Meaningful comments.



Tips:

- **Start early**. Never underestimate how long it takes to make a game! Since you are coming up with a game idea and implementing it from scratch, ***we expect you to do a lot more work***.
- When splitting code between multiple files, put code specific to this game in main.c or other files. Those other files should be specific to a concept (response to collision, a specific state of the state machine, etc.).
- Organize your code into functions specific to what that code does. **Your main function and main.c file should not be very long at all!**
- Make sure the order of calling your functions takes into account waiting for vBlank at the correct times to minimize flicker.
- Make sure to **have a goTo function for transitioning** into a new state that is separate from the main function for each state.
- **Use the HW03 scaffold** (this is required!), and make sure that you **create new .h and .c files for your game** (also required!), e.g. `game.h` and `game.c`.
    - For better organization, we recommend you put all the game-specific functions in this file, e.g. `updateGame()`.
- Reference the Recitation and Lecture files on Canvas to review concepts, and feel free to reach out to the TAs if you have any questions!

Submission Instructions:

Ensure that **cleaning** and building/running your project still gives the expected results.

Please reference previous assignments for instructions on how to perform a "clean" command if you need clarification.

Zip up your entire project folder, including all source files and everything produced during compilation **(including the .gba file)**. Submit this zip on Canvas. Name your submission **HW03_LastnameFirstname**, for example:

> “HW03_TheurerDave.zip”

It is your responsibility to ensure that all the appropriate files have been submitted, and that your submitted zip can be opened and everything cleans, builds, and runs as expected.